

Agentic Engineering: Five Papers

A pragmatic reader's guide — 30 May 2026

From the arXiv daily agentic-engineering digest · 30 May 2026

30 May 2026

Chapter 0: The Environment Became the Product

Five papers landed in the same week of late May 2026, from five different labs, on what look like five different problems: writing verifiable research reports, training agents in a terminal, measuring what agents remember, driving real phones, and making web agents cheaper. Read them together and a single conviction shows through all five. The frontier of agent engineering has moved off the model and onto everything around it — the world the agent acts in, the gates that check its work, the memory it carries, and the honest accounting of what any of it costs.

The clearest signal is that three of these five papers are, at bottom, about building or measuring *environments*. LiteCoder-Terminal scales long-horizon command-line worlds where an agent's keystrokes have consequences. PhoneWorld turns real graphical interfaces into a reusable supply of phone-use tasks. WorldMemArena builds an arena whose entire purpose is to probe memory through action rather than quiz it with static questions. None of these is a new model. The contribution in each case is the world — the thing that produces training signal, the thing that grades

behavior, the thing that exposes a weakness a leaderboard would hide. If 2024 was about better weights and 2025 about better prompts, this batch argues that 2026 is about better environments.

The second thread is a hardening insistence on verification and measurement over raw generation. M-DARE’s central finding is almost blunt about it: pull the verifier out of a multi-agent report writer and the system does not merely get worse, it collapses, with runs dying on malformed tool calls before they produce anything. The acceptance gate is not a finishing polish; it is the load-bearing wall. WorldMemArena makes the same move in evaluation, refusing to credit an agent for reciting a fact and instead asking whether it can act on what it should remember. PANDO extends it to cost, auditing a thousand rollouts to show that a third or more of failures were the same dead button clicked again and again — a defect you only see if you instrument the run rather than trust the score. The common instinct across all three is to stop trusting the chain and start checking it.

Third, persistent structure beats more inference. PANDO’s skill library grows during the test run itself, distilling small reusable routines from what worked so later tasks call them directly instead of rediscovering them — and crucially, it demotes skills that rot, which is what keeps the library from becoming a liability. M-DARE carries a Visual Working Memory that keeps each retrieved image bound to the text it came from, so evidence stays aligned with meaning instead of being matched after the fact. The pattern is the same in both: give the agent durable state that compounds, and you buy more than you would by spending the same tokens on a longer think. Memory and skills are becoming first-class components of the harness, not afterthoughts.

Fourth, and quietly the most consequential, is honest accounting. PANDO’s parting argument is that the field should stop reporting success rate alone and start reporting it next to raw tokens, cached-token share, hidden discovery cost, and tokens per *successful* task — because the same accuracy number can hide wildly different bills. WorldMemArena is the evaluation-side version of the same honesty: measure the thing that actually matters in deployment, not the proxy that is easy to score. A field that is starting to audit its own hidden costs is a field that is growing up.

What to carry into the chapters that follow. Read M-DARE (Chapter 1) for the discipline of acceptance gates — the cheap deterministic check that refuses to pass broken output forward, and the experiment of adding exactly one such gate to a pipeline you already run. Read LiteCoder-Terminal (Chapter 2) and PhoneWorld (Chapter 4) as a matched pair on how to manufacture training and evaluation environments at scale, one in the terminal and one on the screen, and notice what each had to solve to make its world both large and trustworthy. Read WorldMemArena (Chapter 3) for a sharper definition of what “agent memory” should even mean, and a benchmark that will embarrass systems that conflate recall with use. Read PANDO (Chapter 5) for the clearest demonstration yet that an agent can get *cheaper* as it gains experience — and for a cost vocabulary worth adopting whether or not you ever build a skill library.

The five papers do not cite a shared agenda. They arrived at one anyway. The model is no longer the interesting variable; the harness around it is. These chapters are five views of that harness — how to gate it, how to grow the worlds that train it, how to measure what it keeps, and how to make it pay for itself.

Chapter 1: Verifiable Multimodal Deep Research

Paper: [Towards Verifiable Multimodal Deep Research: A Multi-Agent Harness for Interleaved Report Generation](#) · arXiv 2605.29861

Turn off the verifier and the system collapses. When the researchers behind PTAH stripped the verification step out of their report-writing pipeline and ran it across 100 research tasks, 14 never made it out of the planning stage. They died on parsing errors, repetitive loops, and malformed tool calls. Of the 86 that survived, another 18 fell apart during research. Sixty-eight finished. That single ablation is the whole argument: in a multi-agent system that writes long documents, the component that decides whether work is good enough is not a finishing touch. It is the thing holding the machine together.

PTAH is built for a hard version of the report problem. Deep search answers a factual question and stops. Deep research synthesizes scattered evidence into a long, structured document, and the good ones interleave text with pictures: a trend chart next to the paragraph that cites it, a diagram beside the explanation it illustrates. There is no answer key for a five-page report, which means you cannot grade it against ground truth, and a single sloppy retrieval early on quietly poisons everything downstream.

Most systems treat the images as decoration. They write the prose, then paste in figures at the end, so the visual evidence floats loose from the text it supposedly supports. PTAH refuses that ordering. It splits the job across specialized agents and runs them through three stages, with a verifier checking the output of each stage before the next one begins.

The first stage plans. A planner agent searches the topic, then writes a structured outline that names not only the sections but the visual evidence each section needs, where it should sit, and what job it does. Chart, screenshot, diagram, or illustration: the plan commits up front. That plan becomes a working document the rest of the system must honor.

The second stage gathers evidence in parallel. One researcher agent per section goes out, retrieves sources, and assembles a package of claims, numbers, tables, citations, and writing instructions. As it reads each webpage, it also pulls the images off the page and files them into something the paper calls Visual Working Memory: every candidate image stored with its source URL, the surrounding page text, the section it belongs to, and the role it is meant to play. Cheap filters drop blurry or duplicate images; a vision-language model then picks the ones that actually match the section's stated visual need. Because the image arrives already attached to the text that surrounded it on the original page, it comes pre-aligned with its meaning instead of being matched to the prose after the fact.

The third stage writes. Rather than dropping in pictures during cleanup, the writer agent generates prose and image directives together, planting a tag in the text exactly where a visual belongs and declaring how to fill it. Three options: reuse an aligned image already in Visual Working Memory (preferred), search the web for a new one, or generate one outright, running code to render a data chart or calling an image model for an illustration.

Then the system reviews its own draft six ways before showing it to anyone: refine each section, decide keep-delete-edit on every image, fix global consistency, generate HTML, refine the HTML, and render the page in a browser. The paper calls this verifier-guided test-time scaling, and the ablation shows it earns its cost. With it, citation accuracy hits 87.5 percent against 60.7 for the best baseline, and the system carries roughly 9.6 valid citations per report against the field's 2.3. Drop the refinement loop and the report quality falls three points while the count of broken, failed-to-generate images climbs.

For anyone building agent systems, the lesson cuts against a common instinct. The temptation with multi-step agents is to chain models and trust the chain. PTAH's results say the chain is only as stable as its acceptance gates. The verifier here is not one model grading another with vibes; it pairs rule-based checks (is the JSON valid, did it follow the protocol, do the tool calls parse) with rubric-based judgment (does the evidence support the claim, does the image match the text). Cheap deterministic checks catch the failures that kill runs. Expensive judgment checks catch the failures that erode quality.

So here is the experiment to run Monday. Take whatever multi-step agent you already have and add a single deterministic acceptance gate between two stages. Not an LLM judge yet, just a schema validator and a protocol check that refuses to pass malformed output forward, with a bounded retry. PTAH's collapse came overwhelmingly from mechanical failures, and those are the ones a few lines of validation catch for almost nothing. Measure how many runs die before and after. Watch one thing in particular: retries that loop without converging. The verifier feedback in PTAH pushed agents to search more and ground claims harder, but an acceptance function with no retry ceiling can also trap an agent in a rejection cycle, burning tokens on the same rejected output. Cap it, log the rejections, and read them; the rejection log is your cheapest map of where the pipeline actually breaks. Only once the deterministic gate is paying off should you spend tokens on a model-based reviewer for the softer questions of grounding and alignment.

PTAH is about trusting what an agent writes. The next chapter turns to the harder problem underneath it: trusting what an agent does. LiteCoder-Terminal tackles the environments that train language agents to act in the world rather than merely report on it, building scalable, long-horizon command-line settings where an agent's every keystroke has consequences. If verification is how we keep a report honest, environments are how we teach an agent to earn that trust in the first place.

Chapter 2: LiteCoder-Terminal

Paper: [LiteCoder-Terminal: Scaling Long-Horizon Terminal Environments for Learning Language Agents](#) · arXiv 2605.29559 · **Project page:** huggingface.co/Lite-Coder

To teach an agent to live inside a terminal, you first need terminals worth living in. Most teams get them by scraping GitHub: find real repositories, make them runnable, mine their issues for tasks. That works, sort of, until you want a specific skill the scraped pile doesn't happen to cover. You can't order more networking tasks. You take what the internet gave you.

LiteCoder-Terminal-Gen starts from the opposite end. Instead of scraping the world and filtering down, it manufactures practice environments to order, from nothing but a domain name. Tell it you want a data-science task, or a security task, or a build-tools task, and it writes the task, builds the Docker container, writes a reference solution, and writes the test that grades it. No external repository required. The authors call it zero-dependency, and the word is literal: the only input is a list of ten domains.

The trick that gets it started is borrowed from a data-synthesis method called Magpie. Take an aligned language model, hand it a domain-specific system prompt, then append the token that normally begins a user's turn — and stop. The model, primed to continue the conversation, invents the user's request itself. A trailing tag becomes a task generator. A feasibility filter then throws out anything vague, anything needing a GPU or external login, anything a CPU-only container couldn't finish in an hour.

A task description is not an environment, though, and this is where the pipeline earns its keep. Five agents run in sequence, each reading a shared log of everything the earlier ones did. One refiner pins every input and output to a concrete path under /app and nails down the output format. One builds the Dockerfile and the sample data. One writes a reference solution — which doubles as proof the task is actually solvable. One writes the verifier. One sets the resource limits.

The verifier stage is the part most synthetic-data schemes quietly botch. A test written right after the reference solution tends to memorize that one solution and reject every other correct answer. So the verifier agent is forced through a four-phase drill on every assertion. Draft a check. Attack it by simulating a lazy agent that submits an empty file or a hardcoded dummy — if that passes, the check is too weak. Refine it against an imagined expert who solves the task a different valid way — if that gets rejected, the check is too strict. Then finalize. The grader has to survive both a cheater and an honest contrarian before it ships.

Run this and you get two corpora: 11,255 expert trajectories for supervised fine-tuning, averaging 27 turns each, plus 602 fully executable environments for reinforcement learning. The trajectories themselves come from strong teacher models driving real agent scaffolds — Terminus, Claude Code, OpenHands — and then get filtered by a judge that drops any run where the agent looped on the

same command, hallucinated success it never verified, gave up at the first “command not found,” or refused outright. What survives is not just correct, but well-behaved under failure.

The numbers are the argument. A 32B model fine-tuned on this synthetic data hits 29% on Terminal Bench 1.0 and 34% on Terminal Bench Pro, beating its base model by up to 17 points and quadrupling its score on the harder 2.0 benchmark. The comparison that should make you sit up: rival datasets built by mining real repositories used 50,000 and 490,000 trajectories. LiteCoder matched or beat them with 11,000 — up to 44 times fewer. Synthesis from scratch turned out to be more sample-efficient than mining the real thing.

Two findings travel beyond the leaderboard. First, the skills generalized: a model trained only on terminal tasks roughly doubled its score on SWE-bench, a software-engineering benchmark it was never tuned for. Learning to live in a terminal taught something portable about repository work. Second, the verifiers held up under RL — applying preference optimization on the 602 environments pushed the 4B model further on the hardest benchmarks, which only happens if the auto-generated graders are giving honest reward.

So here is the Monday move. If you’re training a terminal or coding agent and you’ve identified a specific weak spot — say the agent fumbles networking or version control — don’t go scrape for more examples. Generate them. Stand up the Magpie-style sampler against that one domain, run the five-stage pipeline, and the adversarial verifier drill is the piece you cannot skip: a grader that hasn’t been attacked by a fake lazy submission will happily reward garbage. Watch two things. The task distribution inherits the generator model’s biases, so a single teacher will quietly narrow what your agent ever sees. And every environment here is Ubuntu plus GNU tools — train only on that and your agent learns one operating system’s habits as if they were universal.

The deeper shift is treating capability gaps as something you can target on demand rather than hope to find in the wild. The bottleneck moves from data you scavenge to environments you author.

Which raises the harder question about any agent that has learned to act across hundreds of these worlds: what does it actually carry from one moment to the next? Solving a 27-turn task means remembering what you tried ten commands ago, what the environment told you, what already failed. Static recall quizzes — can you repeat this fact — miss that entirely. The next chapter turns to WorldMemArena, a benchmark that measures a multimodal agent’s memory the way these environments measure its skill: through acting in a world and seeing what it remembers when the world acts back.

Chapter 3: WorldMemArena

Paper: [WorldMemArena: Evaluating Multimodal Agent Memory Through Action-World Interaction](#) · arXiv 2605.29341 · **Code:** github.com/WorldMemArena/WorldMemArena

An agent is working through a long household task. Early on it logs a budget marked “updated,” a risk list “shared,” a project scope “approved.” Many steps later a question arrives that depends on one of those facts — except the budget was revised twice since, and the scope was renamed. A weak agent answers from the first thing it stored; a strong one knows which version is still true. Most agents we build today answer from the first thing they stored, and that is the failure WorldMemArena is built to catch.

The way the field has measured agent memory is thin. You paste a long transcript into the context window and ask the model to find a fact buried in it, or you score how coherent a chatbot stays over many turns of text. Both treat memory like a filing cabinet: the answer was fixed the moment it went in, and it never changes. Worse, both collapse everything into one end-of-task accuracy number, so when the agent gets it wrong you cannot tell whether it never stored the fact, stored it but never surfaced it, or surfaced a stale copy. And both flatten images into captions before scoring, which quietly excuses visual memory from ever being tested.

WorldMemArena reframes memory as a loop instead of a cabinet. At each step the agent observes a partly visible world, acts, gets feedback, and uses memory to decide what to do next and what is worth keeping. The benchmark is 400 multi-session tasks — roughly 18 sessions and 24,000 question-answer pairs in all — split across two regimes. Lifelong Evolution tracks personal and project states that drift across sessions, so a budget you logged in session two is wrong by session five and has to be revised. Agentic Execution is built from real agent trajectories — GUI and mobile interfaces, plus embodied worlds like OmniGibson and Minecraft — where the useful memory is scattered across observations, tool feedback, and failed actions rather than handed over as tidy narrated text.

The real move is refusing to score memory as a single number. WorldMemArena annotates every session with gold memory points (what should have been kept), state updates (where an old fact goes stale), and distractors (plausible but superseded noise), then grades four observable stages. *Write* asks whether the agent stored the facts worth storing. *Maintain* asks whether it revised the stale ones and dropped the dead ones. *Retrieve* asks whether the right evidence surfaced at decision time. *Use* asks whether that evidence actually shaped the answer. When a task fails, you can point at the stage that broke it.

That decomposition exposes a comforting illusion. Storing more correct memories does not produce better answers. Systems that ace the Write stage — high recall, low hallucination — still flounder at question time, because correct storage is worthless if the right item never gets retrieved or used. One

system built good memories and still landed near the bottom on final accuracy, purely because it could not surface the key evidence when it mattered. Coverage is not capability.

And one stage breaks worse than the rest. Across nearly every system tested, maintenance is the weak link: agents append new information when the world changes but rarely revise, overwrite, or delete the obsolete version. They accumulate instead of maintaining state. The benchmark scores an update as successful only when the new fact is kept *and* the old one is removed — and by that honest standard most systems fail, because both versions sit in memory and the model picks badly between them later. Retrieval-augmented setups are the clearest case: the vector store keeps the old and new observation side by side, and similarity search keeps pulling the stale one. RAG remembers too well. It can add, but it cannot invalidate.

Notice the shape of it. Maintaining state needs three things to go right — the agent has to notice the world changed, locate the belief that is now wrong, and overwrite it rather than append. Almost everything we build is designed to append, because appending is safe: you never lose data. The cost is that old and new coexist, and adjudicating between them is what these models do poorly.

Two more findings will reorder your priorities. Multimodal memory barely works: systems with full visual access score little better than text-only ones, because they compress what they saw into a caption, lose the spatial detail, and fail on visual search and cross-modal reasoning later. If your agent perceives through pixels, assume its memory of what it saw is far weaker than its memory of what it read. And the self-managing agent harnesses — the ones that author and reorganize their own memory during a run, the way coding agents do — are more flexible than hand-built pipelines and often beat them, but they are costly and unstable, with the same model performing very differently under different harnesses.

So here is the experiment to run Monday. Test your agent’s memory adversarially: log a fact, change it mid-run, query it, and check which version the agent acts on. That single contradiction test will tell you more than any needle-in-a-haystack score, because it targets maintenance, the stage that actually fails. If you are on RAG, stop treating recall as the goal and add an explicit invalidation step — when a new observation contradicts a stored one, mark the old entry dead so retrieval can no longer surface it. Score yourself the way WorldMemArena does: an update only counts if the stale copy is gone. And budget for the fact that visual state will not survive the trip into memory on its own; narrate what the agent sees into text it can store and revise, rather than trusting the image to be remembered.

Measuring memory inside a moving world is one thing. Living in one at scale is another, and the next chapter drops into the most demanding interactive world most of us already carry in a pocket. PhoneWorld builds a large environment for training and evaluating agents that drive real smartphone interfaces, where the screen is the world, every tap rewrites the state, and a stale belief about what is on screen carries consequences far more concrete than a misremembered budget.

Chapter 4: PhoneWorld

Paper: [PhoneWorld: Scaling Phone-Use Agent Environments](#) · arXiv 2605.29486

Replace ten thousand training steps with a different ten thousand and a phone agent’s score on a held-out test jumped fifty-two points. No bigger model, no extra data, no longer training run — the same nine-billion-parameter backbone, the same total budget. The only thing that changed was where the training experience came from. That single swap is the argument PhoneWorld makes: the thing holding phone agents back is not the model, it is the supply of environments to train and test them in.

A phone-use agent is a model that drives a smartphone the way a person does — reading the screen, tapping, swiping, typing — with no privileged API into the app. The trouble is that real apps are terrible to train against. They change weekly, they need a network and a login, and you cannot reset them to a known state to run the same task a thousand times. So the field has hand-built one small benchmark at a time, and the agents stayed small with them.

PhoneWorld’s move is to stop copying real apps and start rebuilding them as controllable fakes. A human taps through a real app — QQ, a shopping app, a food-delivery app — in ordinary exploratory use, and the pipeline records the screenshots and the action trace. From that trace it recovers the app’s skeleton: which screens get visited most, how they connect, which taps actually change something. A coding agent then writes a runnable mock of the app in Kotlin, reproducing the screens that matter and skipping the long tail that nobody touches.

The clever part is what sits behind those mock screens. The app’s content — products, messages, listings — lives in a read-only layer so the agent can browse and search realistic data with no network. Everything the agent can change — a cart, a favorite, a sent message — lives in a resettable SQLite database. That split is what turns a fake app into a usable environment: you can reset it to a clean state, run the same task again, and run it ten thousand more times.

That database is also how grading stops needing a human. On a real phone you usually cannot see inside the app to confirm a task succeeded. Here you can just query the SQLite state: did a row appear in the favorites table with the right item id? For information-seeking tasks, check whether the agent’s final answer contains the key value pulled from the read-only content. Both checks are deterministic — no model acting as judge, no run-to-run variance. That is what lets the task pool grow past anything humans could label by hand: PhoneWorld generates nearly eight thousand tasks, runs an agent across all of them, keeps only the verifier-confirmed successes, and harvests thirty-six thousand training steps from the wins.

The current suite is thirty-four mock apps across sixteen domains, and the apps are not built from scratch each time. Eighteen reusable modules — search, feed cards, comments, cart and checkout, messaging — get shared across them, so adding a new app means writing only what is genuinely

app-specific. Every app you add enlarges the test set and the training supply at the same time, from the same infrastructure, with no separate data-collection campaign.

For anyone building computer-use or phone-use agents, the result worth internalizing is which knob moved the needle. More training data helped, but with diminishing returns. What helped far more, under a *fixed* data budget, was breadth: drawing the same ten thousand steps from thirty-four apps instead of five raised every benchmark at once — including the real-app AndroidWorld test the agent had never been trained on. Variety of environments beat volume of trajectories. If your agent is plateauing, the fix is probably not more episodes of the same five apps; it is more different apps.

So the experiment to run Monday is a small version of this loop. Pick one app you care about and define success as a database query, not an internal API call you happen to have — the row that should exist after the task is done. Seed the app’s content as read-only data, put the mutable state in a SQLite file you can snapshot and restore, and write a reset that drops back to the snapshot between runs. Now you can run the same task hundreds of times and grade every run automatically. Watch for the failure mode this design exposes: most phone-agent errors are perception and grounding, not reasoning — the agent knew the plan and tapped the wrong pixel, or missed a modal that slid in. And do not chase pixel-perfect mock fidelity; the paper’s reviewers checked only that the main flows worked and looked close enough to operate on. Faithful-enough and resettable beats photoreal and frozen.

There is a caveat the authors are honest about. Swapping out *all* of the real-app training data for PhoneWorld data spiked in-domain scores but dropped real-app transfer by ten points — the mock apps are selective abstractions, not full replicas, and they cannot replace contact with the genuine article. The strongest setting kept most of the real-app data and replaced only a slice. Synthetic breadth and real grounding are complements, not substitutes.

Build environments this rich and you get agents that are genuinely more capable — and, just as reliably, more expensive, since every screenshot of every step runs through a heavy multimodal model. That cost is the wall between an agent that works in the lab and one you can afford to put on a phone. The next chapter, on PANDO, takes that wall head-on: how to distill the skills a big model demonstrates into a smaller, cheaper one while the agent is actually running — online skill distillation — so the capability you trained in a rich environment survives the trip to deployment.

Chapter 5: PANDO

Paper: [PANDO: Efficient Multimodal AI Agents via Online Skill Distillation](#) · arXiv 2605.24785

For the past two years, the recipe for a smarter AI agent has been the same: spend more tokens. Run the task ten times and keep the best answer. Add a second model to double-check the first. Send the agent off to explore a website for an hour before the clock starts. Every one of those tricks buys accuracy, and every one of them bills you for it on each new task, forever.

PANDO asks a different question. Can an agent get *cheaper* as it gains experience, the way a person does?

The setting is VisualWebArena, a benchmark of 910 real web tasks across classifieds, shopping, and Reddit. The agent looks at screenshots, clicks buttons, fills forms. When the researchers audited a thousand baseline rollouts, they found the waste was not random. Between a third and 42 percent of failures were repeat-action loops, the agent clicking the same dead button over and over. Another chunk was hidden discovery cost, hours of off-benchmark exploration that headline scores quietly omit. And the prompts barely reused their own cache, under 11 percent for text-based methods, so the same instructions got recomputed from scratch every step.

These are mechanical inefficiencies, not “the model is dumb” problems. So PANDO attacks them with persistent structure on the agent’s side rather than more inference.

The center of the system is a Skill Library that grows during the test run itself. It holds two kinds of things. Rules are guardrails: if you just clicked this and the page did not change, stop, try something else. Routines are small reusable programs, like `apply_price_filter(min, max)` or `sort_by_attribute(attr, dir)`, that collapse a five-click subgoal into one deterministic call. After each successful task, a learning module distills new routines from what worked and admits them to the library, so the next task can call them directly.

The clever part is what keeps the library from rotting. Most “agents that learn skills” only add, never subtract, so they bloat with brittle routines that silently stopped working. PANDO tracks a confidence score for every skill, and once one fails enough times it goes on a demotion blacklist. Those failed signatures then block 36 attempts to rediscover the same broken skill. It also merges mirror-image routines, “cheapest” and “most expensive” become one routine with a direction flag, which doubles how often the routine matches a new task.

Retrieval is deliberately dumb. Skills are looked up by literal keyword matching, not by embedding similarity. That sounds like a step backward until you see why: a literal index is auditable, deterministic, and, above all, it keeps the prompt prefix stable so the cache stays warm.

The numbers are where this turns from tidy to compelling. On all 910 tasks, PANDO hits 58.3 percent success, ahead of the strongest prior single-rollout system (SGV at 54.0) and well past their WALT reproduction (45.2). It does this while spending 115,000 tokens per task against SGV’s 275,000 and

WALT's 294,000, roughly 58 to 61 percent fewer. It is the only system on the chart that is both more accurate and cheaper than everything else. Steps per task drop to 9.3, the lowest of any method and not far above the human's 7.7.

What makes the efficiency real rather than cosmetic is that it compounds over the run. The first hundred tasks are a rough cold start, around 10.6 steps each and only 62 percent cache reuse. By the final 310 tasks, steps fall to 8.9, tokens to 103,000, cache utilization climbs to 76 percent, and more than half of tasks fire a learned skill. The agent is not just good; it is getting cheaper as the stream proceeds.

An ablation tells you which parts do which job, and this is the practical lesson. The skill components, rules, seed routines, the progress-checking reflector, online distillation, and demotion, supply almost all the accuracy, lifting success from 38.6 to 57.3 percent. The remaining pieces, routing cheap work to a cheap model, compressing screenshots, and cache-aware prompt layout, add barely two points of accuracy but do most of the token cutting. Competence and cost are separable. You can adopt one without the other.

For anyone deploying a multimodal agent and watching the bill, the order of operations is clear. Start with a loop-detection rule and a handful of seed routines for your most repeated subgoals, before reaching for any clever prompting. That alone bought the biggest single jump in their data. Then add a confidence counter and a demotion list, because the experiment to run Monday is the one that proves your library needs pruning: let routines accumulate without demotion and measure your repeat-action rate over a few hundred tasks. PANDO's audit showed stale routines quietly drive that rate up, turning your "learning" into a hidden token tax.

Two cautions travel with the result. Everything here is web tasks; desktop work with pixel-level misclicks and multiple windows will need new rules they have not written. And online learning assumes a trusted task stream, an adversary feeding tasks in a hostile order could inflate that cold-start cost. The residual failures also point the way forward: loops are no longer the main problem, grounding errors are, so the next gains come from better clicking, not longer reasoning.

The wider point outlasts the benchmark. The field has treated tokens spent on a task as money burned. PANDO treats them as capital, work done once that lowers the cost of every task after it. Its parting suggestion to the rest of the field is modest and overdue: stop reporting success rate alone. Report it alongside raw tokens, cached-token share, hidden discovery cost, latency, and tokens per successful task, because the same accuracy number can hide wildly different bills. An agent that remembers is worth more than one that merely scores well, and now there is a system that shows what remembering is worth.